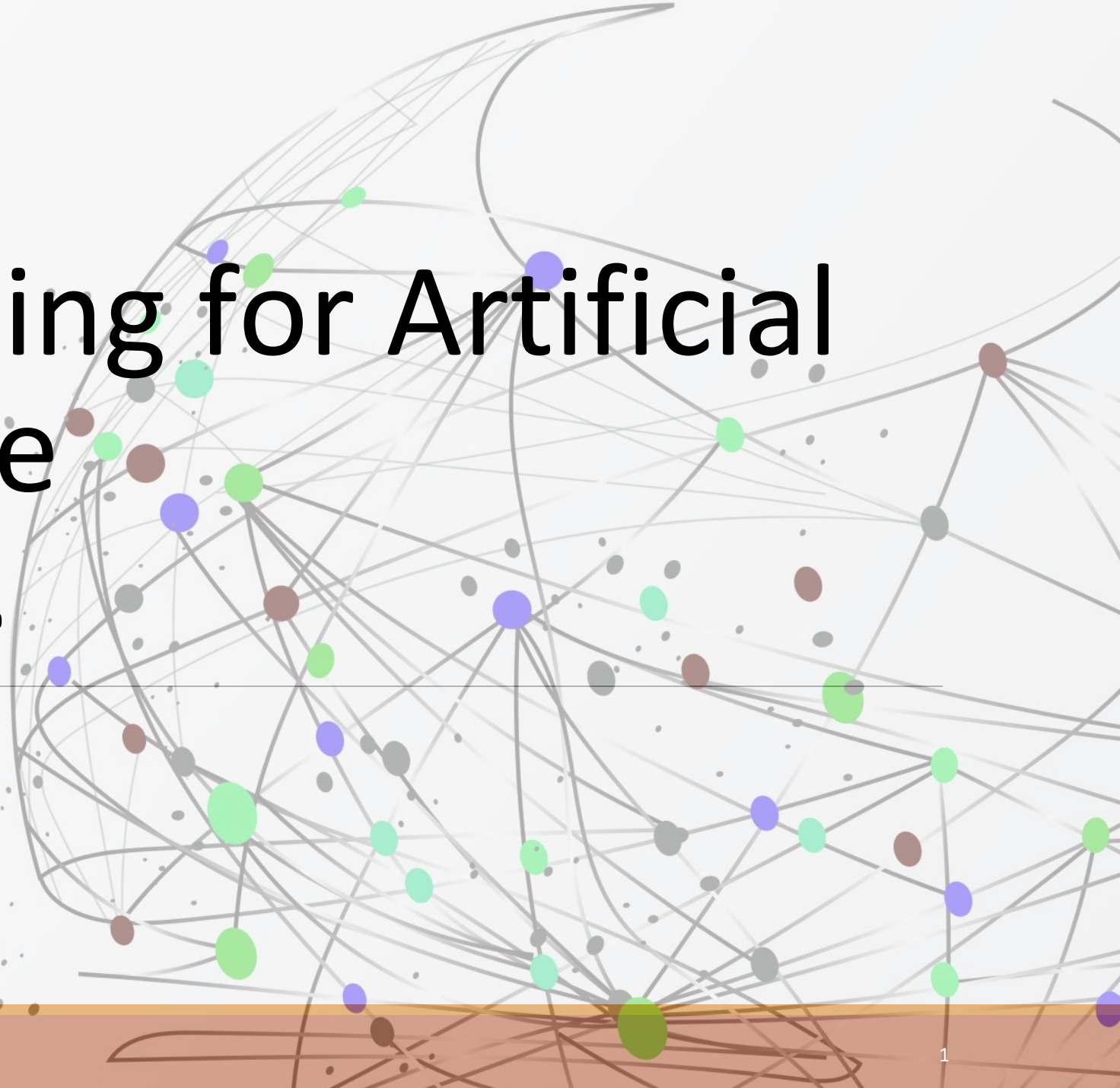
Programming for Artificial Intelligence
Lecture# 2

BY: MS. SALMA ASIF



Outline

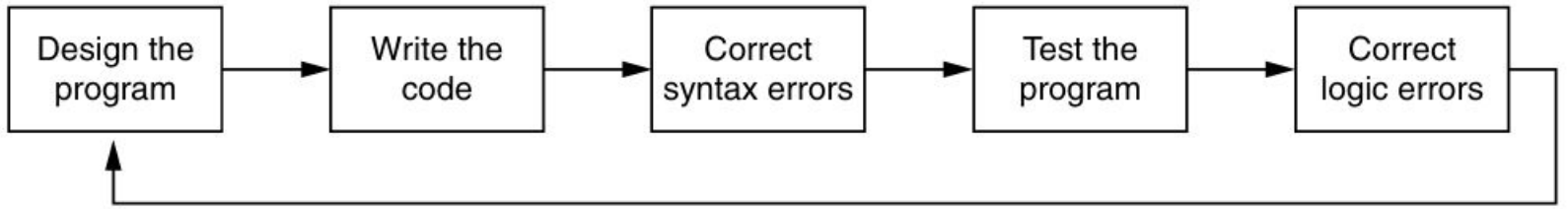
- Designing a Program
- Input, Processing, and Output
- Displaying Output with the print Function
- Comments
- Variables
- Reading Input from the Keyboard
- Performing Calculations
- More About Data Output
- Typecasting

Designing a Program

- Programs must be carefully designed before they are written. During the design process, programmers use tools such as pseudocode and flowcharts to create models of programs.

Program Development Cycle

Figure 2-1 The program development cycle



Designing a Program

- Suppose you have been asked to write a program to calculate and display the gross pay for an hourly paid employee.
  1. Get the number of hours worked.
  2. Get the hourly pay rate.
  3. Multiply the number of hours worked by the hourly pay rate.
  4. Display the result of the calculation that was performed in step 3.

- An **algorithm**, which is a set of well-defined logical steps that must be taken to perform a task.

Designing a Program

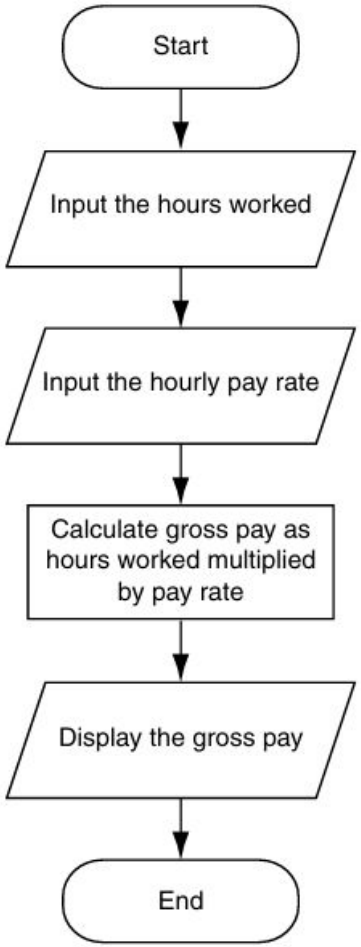
- Programmers write a program in **pseudocode** before they write it in the actual code of a programming language such as Python.
- The word “pseudo” means fake, so pseudocode is fake code.
- It is an informal language that has no syntax rules and is not meant to be compiled or executed.

e.g;

1. Input the hours worked
2. Input the hourly pay rate
3. Calculate gross pay as hours worked multiplied by pay rate
4. Display the gross pay

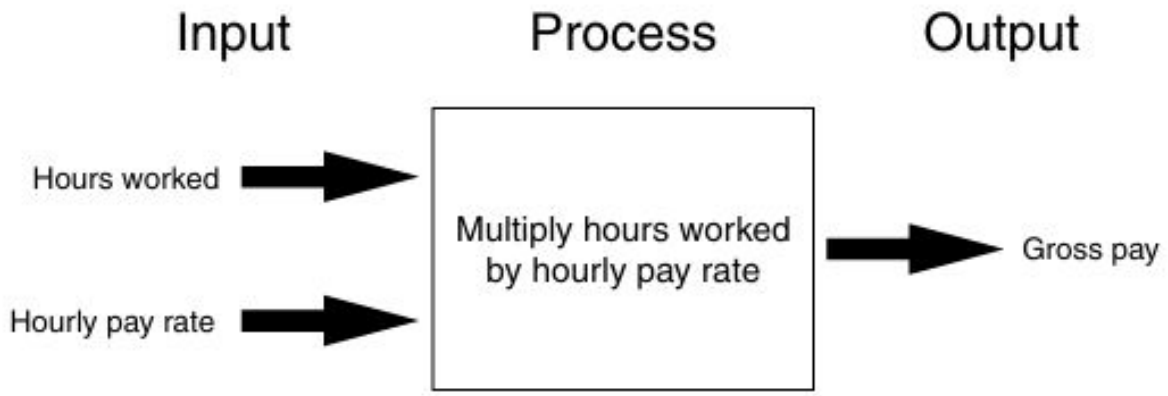
Designing a Program

- A flowchart is a diagram that graphically depicts the steps that take place in a program.
- There are three types of symbols in the flowchart: **ovals**, **parallelograms**, and a **rectangle**.
  - The **ovals**, which appear at the top and bottom of the flowchart, are called terminal symbols.
  - **Parallelograms** are used as input symbols and output symbols. They represent steps in which the program reads input or displays output.
  - **Rectangles** are used as processing symbols. They represent steps in which the program performs some process on data, such as a mathematical calculation.
- The symbols are connected by arrows that represent the "flow" of the program.



Input, Processing, and Output

Figure 2-3 The input, processing, and output of the pay calculating program



Displaying Output with the **print** Function

- Use the **print** function to display output in a Python program.

```
print("Hello, World!")
```

```
>>> print('Hello world')Enter
Hello world
>>>
```

Displaying Multiple Items with the **print** Function

Program 2-9 (variable_demo3.py)

```
1  # This program demonstrates a variable.
2  room = 503
3  print('I am staying in room number', room)
```

Program Output

```
I am staying in room number 503
```

Strings and String Literals

'Kate Austen’

'123 Full Circle Drive ‘

'Asheville, NC 28899’

- Sequences of characters
- In programming terms, a sequence of characters that is used as data is called a string.
- When a string appears in the actual code of a program, it is called a string literal
- string literals in a set of single-quote marks (') or a set of double-quote marks (") or triple quotes (either """ or '’’)
- Triple quotes can also be used to surround multiline strings, something for which single and double quotes cannot be used

Strings and String Literals

Program 2-2 (double_quotes.py)

```
1 print("Kate Austen")
2 print("123 Full Circle Drive")
3 print("Asheville, NC 28899")
```

Program Output

```
Kate Austen
123 Full Circle Drive
Asheville, NC 28899
```

Program 2-3 (apostrophe.py)

```
1 print("Don't fear!")
2 print("I'm here!")
```

Program Output

```
Don't fear!
I'm here!
```

Program 2-4 (display_quote.py)

```
1 print('Your assignment is to read "Hamlet" by tomorrow.')
```

Program Output

```
Your assignment is to read "Hamlet" by tomorrow.
```

Strings and String Literals

```
print("""I'm reading "Hamlet" tonight.""")
```

This statement will print

```
I'm reading "Hamlet" tonight.
```

```
print("""One
Two
Three""")
```

This statement will print

```
One
Two
Three
```

Comments

- Comments are notes that explain the lines or sections of a program.
- Comments are part of the program, but the Python interpreter ignores them.
- Comments: Single-line (#) and multi-line ("""...""") comments

Examples:

```
# This is a single-line comment
    print("Hello, World!")
```

```
"""
This is a multi-line comment.
It can span multiple lines.
It's often used for documentation or
longer comments.
"""
print("Hello, World!")
```

Variables

- **Variables**: Name that represents a value stored in computer memory
- Used to remember, access and manipulate data stored in memory
- Variable references the value
- Assignment statement - used to create a variable and make it reference the desired data
- General format is:

variable_name = value or expression

- Expression is the value, mathematical equation or function that results in a value

Variables

- In assignment statement, variable receiving value must be on left side
- You cannot use a variable UNTIL a value is assigned to it
- A variable can be passed as an argument to a function, like print
- Do not enclose variable name in quotes

Variables

age = 25

width = 10

length = 5

```
>>> print(width)
10
>>> print(length)
??
```

Variables

>>> print(“width”)

??

>>> print(width)

??

Variable Naming Rules

- cannot use Python's keywords as a variable name.
- A variable name cannot contain spaces.
- The first character must be one of the letters a through z, A through Z, or an under score character (_).
- After the first character you may use the letters a through z or A through Z, the digits 0 through 9, or underscores.
- Uppercase and lowercase characters are distinct. This means the variable name **ItemsOrdered** is not the same as **itemsordered**.

Legal and illegal variable names

Variable Name	Legal or Illegal?
`dayOfWeek`	
`3dGraph`	
`_employee_num`	
`June1997`	
`Mixture#3`	

Valid and Invalid Identifiers

IDENTIFIER	VALID?	REASON IF INVALID
`totalSales`		
`total_Sales`		
`total.Sales`		
`4thQtrSales`		
`totalSale$`		

Assignment in Python

The assignment operator in Python is =.

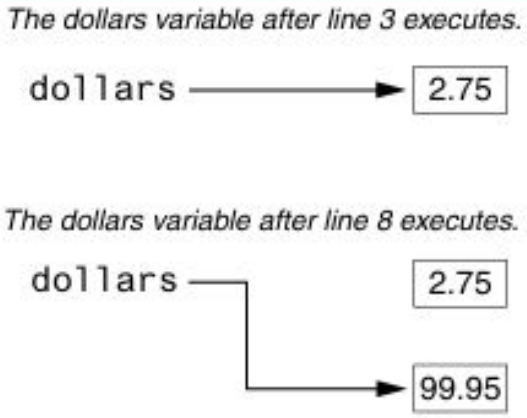
Assigning a value to a new variable creates the variable:

```
#Assigning values to a variable
x = 10
name = "John"
is_student = True

#print the variable
print(x, name, is_student)
```

Variable reassignment in Python

Figure 2-6 Variable reassignment in Program 2-10



Different datatype Variable reassignment

```
1  >>> x = 99 Enter
2  >>> print(x) Enter
3  99
4  >>> x = 'Take me to your leader' Enter
5  >>> print(x) Enter
6  Take me to your leader.
7  >>>
```

Data Types

Python is a dynamically typed language, so we do not need to specify the type of a variable when we create one.

```
# integers
x = 1
print(type(x))
```

```
<class 'int'>
```

```
# float
x = 1.0
print(type(x))
```

```
<class 'float'>
```

```
# boolean
b1 = True
b2 = False

print(type(b1))
```

```
<class 'bool'>
```

```
# complex numbers: note the use of `j` to specify the imaginary part
x = 1.0 - 1.0j
print(type(x))
```

```
<class 'complex'>
```

```
print(x)
```

```
(1-1j)
```

```
print(x.real, x.imag)
```

```
1.0 -1.0
```

Numeric Data Types and Literals

- When an integer is stored in memory, it is classified as an int, and when a real number is stored in memory, it is classified as a float.
- Python interpreter determines data type of numeric literal on the following rules:
  - A numeric literal that is written as a whole number with no decimal point is considered an int. Examples are 7, 124, and −9.
  - A numeric literal that is written with a decimal point is considered a float. Examples are 1.5, 3.14159, and 5.0
- A number that is written into a program's code is called a numeric literal.

e.g;

```
room = 503

dollars = 2.75
```

Strings Data Types

- Python also has a data type named str, which is used for storing strings in memory.

e.g;

Program 2-11 (string_variable.py)

```
1 # Create variables to reference two strings.
2 first_name = 'Kathryn'
3 last_name = 'Marino'
4
5 # Display the values referenced by the variables.
6 print(first_name, last_name)
```

Program Output

```
Kathryn Marino
```

Data Types

```
x= 5
print (type(x) is int)
```

```
True
```

```
x= 5.0
print (type(x) is int)
```

```
False
```

We can also use the `isinstance` method for testing types of variables:

```
print (isinstance(x, float))
```

```
True
```

Reading Input from the Keyboard

- Programs commonly need to read input typed by the user on the key board.
- The Python function **input** used to take input from the keyboard.

variable = input(prompt)

Example:

name = input('What is your name? ‘)

- The value entered by the user is stored in the variables name
- The input function always returns the user’s input as a string,

even if the user enters numeric data.

Program 2-12 (string_input.py)

```
1  # Get the user's first name.
2  first_name = input('Enter your first name: ')
3
4  # Get the user's last name.
5  last_name = input('Enter your last name: ')
6
7  # Print a greeting to the user.
8  print('Hello', first_name, last_name)
```

Program Output (with input shown in bold)

```
Enter your first name: Vinny Enter
Enter your last name: Brown Enter
Hello Vinny Brown
```

Reading Input from the Keyboard

```
# Ask the user for their name
name = input("Enter your name: ")

# Ask the user for their age
age = input("Enter your age: ")

# Output a greeting message
print("Hello, " + name + "!")

# Output a message with the user's age
print("You are " + age + " years old.")
```

```
Enter your name:  Ali
Enter your age:  17
Hello, Ali!
You are 17 years old.
```

Reading Numbers with **input** Functions

- Suppose you call the input function, type the number 72, and press the Enter key. The value that is returned from the input function is the string '72'.

Table 2-2 Data conversion functions

Function	Description
`int(item)`	You pass an argument to the `int()` function and it returns the argument's value converted to an `int`.
`float(item)`	You pass an argument to the `float()` function and it returns the argument's value converted to a `float`.

Reading Numbers with **input** Functions

```
string_value = input('How many hours did you work? ‘)
hours = int(string_value)
```

OR

```
hours = int(input('How many hours did you work? ‘))
pay_rate = float(input('What is your hourly pay rate? '))
```

Simple Input and Output

input() function:The input() function is used to take input from the user.

The string inside the parentheses (e.g., "Enter your name: ") is displayed as a prompt to the user.

Storing Input:The value entered by the user is stored in the variables name

Performing Calculations in Python

- operators are used to perform mathematical calculations

Table 2-3 Python math operators

Symbol	Operation	Description
+	Addition	Adds two numbers
-	Subtraction	Subtracts one number from another
*	Multiplication	Multiplies one number by another
/	Division	Divides one number by another and gives the result as a floating-point number
//	Integer division	Divides one number by another and gives the result as a whole number
%	Remainder	Divides one number by another and gives the remainder
**	Exponent	Raises a number to a power

Operators and comparisons

Arithmetic operators +, -, *, /, // (integer division), '**' power

```
print  (1 + 2, 1 - 2, 1 * 2, 1 / 2)
```

```
3 -1 2 0.5
```

```
print  (1.0 + 2.0, 1.0 - 2.0, 1.0 * 2.0, 1.0 / 2.0)
```

```
3.0 -1.0 2.0 0.5
```

```
# Integer division of float numbers
print  (3.0 // 2.0)
```

```
1.0
```

```
# Note! The power operators in python isn't ^, but **
print (2 ** 2)
```

```
4
```

Arithmetic operators: + – * / %

`a+b, a-b, a*b, a/b`

- Integer division: **`11/4`** gives 2
- Floating point division:
- **`11.0/4`** gives 2.75

Modulo operator: **`%`** (only for integers)

- **`11%4`** gives 3
- **`10%5`** gives 0

Operator	Meaning	Type	Example
+	Addition	Binary	`total = cost + tax;`
-	Subtraction	Binary	`cost = total - tax;`
*	Multiplication	Binary	`tax = cost * rate;`
/	Division	Binary	`salePrice = original / 2;`
%	Modulus	Binary	`remainder = value % 3;`

Floating-Point (/) and Integer Division (//)

- The / operator gives the result as a floating-point value, and the // operator gives the result as a whole number.

```
>>> 5 / 2 Enter

2.5
```

- The // operator works like this:
  - When the result is positive, it is truncated, which means that its fractional part is thrown away.
  - When the result is negative, it is rounded away from zero to the nearest integer.

```
>>> 5 // 2 Enter          and    >>> −5 // 2 Enter

2                               −3
```

Relational Operators

Table 3-1 Relational operators

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

Relational Operators

Table 3-2 Boolean expressions using relational operators

Expression	Meaning
`x > y`	Is x greater than y?
`x < y`	Is x less than y?
`x >= y`	Is x greater than or equal to y?
`x <= y`	Is x less than or equal to y?
`x == y`	Is x equal to y?
`x != y`	Is x not equal to y?

Relational Operators

```
print  2 > 1, 2 < 1
```

```
True False
```

```
print  2 > 2, 2 < 2
```

```
False False
```

```
print 2 >= 2, 2 <= 2
```

```
True True
```

Logical Operators

The logical operators are spelled out as the words **and, not, or**.

Table 3-3 Logical operators

Operator	Meaning
`and`	The `and` operator connects two Boolean expressions into one compound expression. Both subexpressions must be true for the compound expression to be true.
`or`	The `or` operator connects two Boolean expressions into one compound expression. One or both subexpressions must be true for the compound expression to be true. It is only necessary for one of the subexpressions to be true, and it does not matter which.
`not`	The `not` operator is a unary operator, meaning it works with only one operand. The operand must be a Boolean expression. The `not` operator reverses the truth of its operand. If it is applied to an expression that is true, the operator returns false. If it is applied to an expression that is false, the operator returns true.

Logical Operator **and**

Table 3-5 Truth table for the and operator

Expression	Value of the Expression
true and false	false
false and true	false
false and false	false
true and true	true

Logical Operator **or**

Table 3-6 Truth table for the or operator

Expression	Value of the Expression
true or false	true
false or true	true
false or false	false
true or true	true

Logical Operator **not**

Table 3-7 Truth table for the not operator

Expression	Value of the Expression
not true	false
not false	true

Logical Operators

The logical operators are spelled out as the words **and, not, or**.

Table 3-4 Compound Boolean expressions using logical operators

Expression	Meaning
`x > y and a < b`	Is x greater than y AND is a less than b?
`x == y or x == z`	Is x equal to y OR is x equal to z?
`not (x > y)`	Is the expression x > y NOT true?

Logical Operators

The logical operators are spelled out as the words **and, not, or**.

```
print  (True and False)
```

```
False
```

```
print (not False)
```

```
True
```

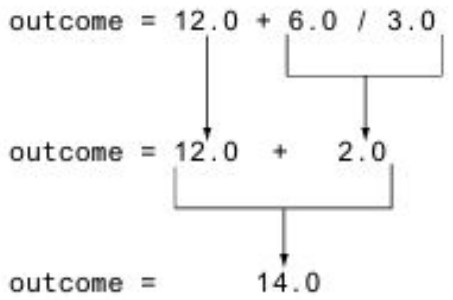
```
print (True or False)
```

```
True
```

Precedence

- First, operations that are enclosed in parentheses are performed first.
- Then, when two operators share an operand, the operator with the higher precedence is applied first.
- The precedence of the math operators, from highest to lowest, are:
  1. Exponentiation: **
  2. Multiplication, division, and remainder: * / // %
  3. Addition and subtraction: + −

Figure 2-9 Operator precedence



Mixed-Type Expressions and Data Type Conversion

- An expression that uses operands of different data types is called a mixed-type expression.
- Python follows these rules when evaluating mathematical expressions:
  - When an operation is performed on two int values, the result will be an int.
  - When an operation is performed on two float values, the result will be a float.
  - When an operation is performed on an int and a float, the int value will be temporarily converted to a float and the result of the operation will be a float.

Breaking Long Statements into Multiple Lines

- Python allows you to break a statement into multiple lines by using the line continuation character, which is a backslash (\).

```
result = var1 * 2 + var2 * 3 + \
         var3 * 4 + var4 * 5
```

- Python also allows you to break any part of a statement that is enclosed in parentheses into multiple lines without using the line continuation character.

```
total = (value1 + value2 +
         value3 + value4 +
         value5 + value6)
```

Solved Example

expr = 7 + 3 ** 2 % 4 * 5 // 2 - 6

1. **Exponents first (`**`)**

   `3 ** 2 = 9`

   Expression → `7 + 9 % 4 * 5 // 2 - 6`

2. **Modulo (`%`)**

   `9 % 4 = 1`

   Expression → `7 + 1 * 5 // 2 - 6`

3. **Multiplication and Floor Division (left to right)**

   `1 * 5 = 5`

   `5 // 2 = 2`

   Expression → `7 + 2 - 6`

4. **Addition & Subtraction (left to right)**

   `7 + 2 = 9`

   `9 - 6 = 3`

Expression	Value
6 + 3 * 5	_____
12 / 2 - 4	_____
9 + 14 * 2 - 6	_____
(6 + 2) * 3	_____
14 / (11 - 4)	_____
9 + 12 * (8 - 3)	_____

More about data output

- The print Function’s Ending Newline
  - print('One’)
  - print('Two’)
  - print('Three’)
- If you do not want the print function to start a new line of output when it finishes displaying its output, you can pass the special argument end=' ' to the function, as shown in the following code:
  - print('One', end=' ‘)
  - print('Two', end=' ‘)
  - print('Three’)

  Output: One Two Three

Specifying an Item Separator

- When multiple arguments are passed to the print function, they are automatically separated by a space when they are displayed on the screen.

>>> print('One', 'Two', 'Three') Enter

Output: One Two Three

- If you do not want a space printed between the items, you can pass the argument sep=‘ ' to the print function, as shown here:

>>> print('One', 'Two', 'Three', sep='') Enter

Output: OneTwoThree

Escape Characters

- An escape character is a special character that is preceded with a backslash (\), appearing inside a string literal.

Table 2-8 Some of Python's escape characters

Escape Character	Effect
`\n`	Causes output to be advanced to the next line.
`\t`	Causes output to skip over to the next horizontal tab position.
`\'`	Causes a single quote mark to be printed.
`\"`	Causes a double quote mark to be printed.
`\\`	Causes a backslash character to be printed.

Escape Characters Examples

- print('One\nTwo\nThree’)
- print('Mon\tTues\tWed’)
- print('Thur\tFri\tSat’)
- print("Your assignment is to read \"Hamlet\" by tomorrow.")
- print('I\'m ready to begin.’)
- print('The path is C:\\temp\\data.')

Displaying Multiple Items with the + Operator

- + operator performs string concatenation.

```
print('This is ' + 'one string.’)
```

Formatting Numbers

- When a floating-point number is displayed by the print function, it can appear with up to 12 significant digits.
- built-in format function.
- The format specifier is a string that contains special characters specifying how the numeric value should be formatted

```
format(12345.6789, '.2f’)

print(format(12345.6789, '.1f'))
```

Formatting in Scientific Notation

- display floating-point numbers in scientific notation

```
>>> print(format(12345.6789, 'e')) Enter
1.234568e+04
>>> print(format(12345.6789, '.2e')) Enter
1.23e+04
```

Inserting Comma Separators

- If you want the number to be formatted with comma separators, you can insert a comma into the format specifier

```
>>> print(format(12345.6789, ',.2f')) Enter
```

12,345.68

Formatting a Floating-Point Number as a Percentage

- you can use the % symbol to format a floating point number as a percentage.

>>> print(format(0.5, '%')) Enter

50.000000%

specifies 0 as the precision:

>>> print(format(0.5, '.0%')) Enter

50%

Formatting Integers

- when writing a format specifier that will be used to format an integer:
  - You use d as the type designator.
  - You cannot specify precision.

```
>>> print(format(123456, 'd')) Enter
123456
```

Specifying a Minimum Field Width

- The format specifier can also include a minimum field width, which is the minimum number of spaces that should be used to display the value.

```
>>> print('The number is', format(12345.6789, '12.2f')) Enter
The number is     12345.68
>>>
```

Type casting

Type casting, or type conversion, refers to the process of converting a variable from one data type to another. In Python, this can be done using built-in functions.

- int() → convert to integer
- float() → convert to float
- str() → convert to string
- list() → convert to list
- tuple() → convert to tuple
- bool() → convert to boolean

Python Syntax and Structure

Indentation: Python uses indentation to define code blocks

```
for i in [1, 2, 3, 4, 5]:
    print i                          # first line in "for i" block
    for j in [1, 2, 3, 4, 5]:
        print j                      # first line in "for j" block
        print i + j                  # last line in "for j" block
    print i                          # last line in "for i" block
print "done looping"
```

Note: This makes Python code very readable, but it also means that you have to be very careful with your formatting.

Questions?